AI Incident Report Assistant: speak it, the QIDK files it

Megathon X · Qualcomm Problem Statement 1 · Team: Shubham Gupta, Sarthak Chaudhari · 11 Oct 2026

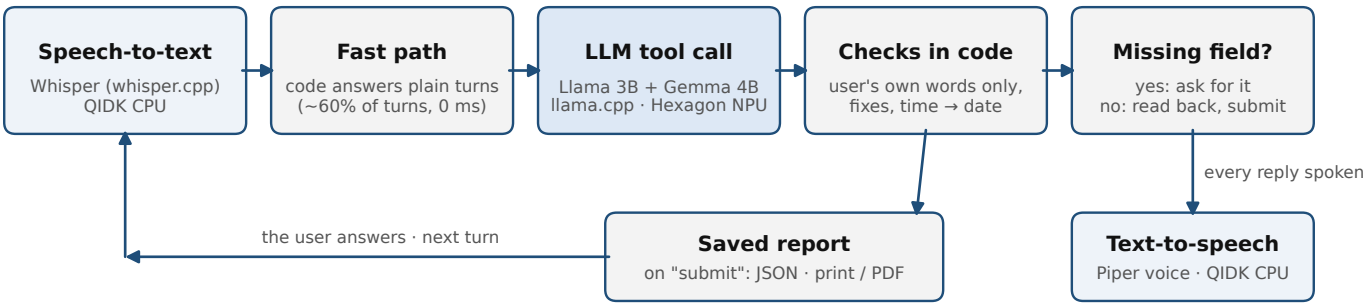
1. Solution

A worker describes an incident in their own words ("There was a water leak near Conference Room 2 around 10 AM. Maintenance has already isolated the area.") and gets a complete, structured report: **speak → understand → extract → review → submit.** The app fills the problem statement's **10-field form** (15 incident types), asks only for what is still missing, reads the report back, takes corrections by voice or click, and saves it. **Every AI model runs on the Qualcomm QIDK (Snapdragon 8 Elite, SM8750P): the language models on the Hexagon NPU, speech on the board's CPU. No cloud, no internet, the data never leaves the device.**

On the board: 95.5% of conversations reach the right outcome (42/44), the action is right on 100% of turns (157/157), and the 4 mandatory fields are right 93.0% of the time (159/171).

The core idea: the model extracts, code decides. One grammar-constrained LLM call per turn turns the utterance into a tool call; plain code checks every value against what the user actually said, decides which mandatory field is missing, resolves times, picks follow-up actions from a catalogue and writes every sentence the user hears. The model can only fill the form, never invent it.

2. Architecture



stage	model (all open weights)	runtime	runs on
Speech-to-text	Whisper base.en (base multilingual for Hindi / Hinglish)	whisper.cpp server	QIDK CPU
Extraction	**Llama 3.2 3B Instruct**, pure Q4_0 (every matrix Q4_0, embedding Q8_0)	llama.cpp b11371, Hexagon backend	**QIDK NPU: 29/29 layers on HTP0** (1,911.9 MiB)
Incident type	**Gemma 3 4B Instruct**, pure Q4_0, first description only	second llama-server, --swa-full	**QIDK NPU: 35/35 layers on HTP0** (2,402.8 MiB)
Text-to-speech	Piper en_US lessac medium	sherpa-onnx	QIDK CPU
Wake word (optional)	Whisper, or a 3.3M-parameter sherpa-onnx keyword spotter	sherpa-onnx	QIDK CPU
Dialog, checks, UI	none: plain Python, standard library only	HTTP server + web page	laptop (drives the board over USB)

NPU placement is proved on every launch from llama-server's own startup log ("offloaded N/N layers", nonzero HTP0 buffer); the page shows it live. The scripts refuse any model file whose sha256 differs from the allow-list.

3. Engineering for the NPU

technique	effect (BOARD unless marked)
Pure Q4_0 weights, all layers on HTP0	29/29 layers on the NPU; published "Q4_0" files carry other tensor types and are refused
Prompt caching	only the new turn is read: ~38–60 new tokens, ~105 ms per call
Fast path	LLM time per turn −37% at the same accuracy (v2)
Speech made after the form is shown	the form updates as soon as the model answers; the voice follows
Answer pre-fill, speculative decoding, shorter type answer, parallel type model	built; ESTIMATE −0.35 s to −1.0 s on the first turn; board runs pending
Heat	NPU peak 79–81 °C, CPU ~100 °C; runs pause at 90 °C NPU (one board reboot at 105.8 °C early on)

4. Approach

- **One agent, one LLM call per turn, grammar on every call.** The model can only emit a valid tool call (update_report / submit_report / cancel_report / no_op) with one-letter keys for the fields allowed; llama-server enforces the JSON schema while decoding, so the output always parses.
- **Output length is the cost on the NPU.** Writing tokens is ~91% of a model call's time (BOARD: ~48 ms per token, prompt read ~105 ms thanks to prompt caching). So the model writes a short tool call, the fixed part of the prompt stays cached, and code writes every sentence.
- **Code where code is enough (the fast path).** Bare answers ("medium", "submit", "near the gate", "around 3 PM", "haan, submit kar do") are answered by code with no model call: 93 of 157 dev turns, never wrong on the eval set.
- **The model never has the last word.** Every free-text value must appear in what the user said (a support check); post-model fixes drop repeats, values in the wrong field, injuries listed as people, complete actions to the end of their clause; times like "around 10 AM yesterday" are resolved by code. Severity is only taken from the user's words (a guess from the story matched the user in 20 of 52 dialogs, so the app asks).
- **Two models, each where it is better (router).** On the same board run, Gemma 3 4B chose the incident type right 39/43 times vs Llama's 33, but Llama was better at everything else and faster. So Gemma decides only the type, once per report; Llama does the rest.
- **Measured, not guessed.** Every change is a version with a keep/drop rule written **before** its board run (eval/DECISION_*.md); numbers are labelled BOARD, LAPTOP, PROJECTION or ESTIMATE; a locked test split is kept for the final check; the board is cooled to ≤ 50 °C before each run and paused at 90 °C.

5. Features

feature	what the user gets
Voice in, voice out	hold the button (or Space) and speak; replies are spoken (Piper) and shown
"Hey Qualcomm" hands-free	say the wake word instead of holding the button; after each question just answer; detected on the board
Live form	the 10 fields fill in as you talk; missing mandatory fields are highlighted; times show the resolved date
Asks only what is missing	type → location → time → severity; severity by voice, everyday words ("pretty serious") or one tap
Review and correct	spoken read-back; correct by voice ("no, it was 11") or click any field
Report view	the submitted report as a clean page: Print / Save as PDF, Download JSON
Follow-up actions	recommended next steps per incident type, from a fixed catalogue (never invented)
Hindi / Hinglish	speak Hindi or Hinglish; translated to English on the device; Hindi labels on the page
Instant feedback	a short spoken "Got it." while the model works on a long turn
On-device proof	layers on HTP0 from the board's log, per-stage latency with the device that ran it, every tool call and every value the checks dropped

6. Results (BOARD: QIDK SM8750P)

Eval set: 65 hand-written dialogs from the problem's example and real incident shapes; dev split = 44 dialogs, 157 turns, 242 field values. Each turn is scored from the correct form (teacher-forced) and each conversation is also played end to end (free-running). 10 more dialogs were written later (54 in all, 309 values). Configuration: v9, Llama 3.2 3B + Gemma 3 4B for the type, one board run each.

metric	44 dev dialogs	all 54 dialogs
Conversations that reach the right outcome (report submitted, or cancelled when the user cancels)	**42 / 44 = 95.5%**	**51 / 54 = 94.4%**
Right action on every turn (update / submit / cancel / nothing)	**157 / 157 = 100%**	**206 / 206 = 100%**
Mandatory fields right (incident type, severity, location, date & time)	**159 / 171 = 93.0%**	**196 / 211 = 92.9%**
severity · location · date & time · incident type	100% · 93.2% · 92.9% · 86.0%	100% · 90.7% · 94.2% · 86.8%
All 10 fields, including the optional lists (persons, equipment, actions, witnesses)	202 / 242 = 83.5%	250 / 309 = 80.9%
Reports with every field exactly right (end to end)	10 / 44	10 / 54

Model time per call: ~1.7 s on later turns, +1.35 s once per report for the type model; ~60% of turns are answered by code with no model call. Speech-to-text and text-to-speech timing on the board: not measured yet (section 10).

7. Accuracy over the versions (BOARD, 44 dev dialogs)

version (what changed)	fields right	tool right	conversations ended right	model time / call
v3: one model + fast path + severity fallback	165 / 242	153 / 157	34 / 44	1.65 s
v4: + post-model fixes	190 / 242	153 / 157	41 / 44	1.72 s
v7: + Gemma decides the incident type	193 / 242	153 / 157	42 / 44	2.68 s*
v9: + list fixes, tool = the app's action	**202 / 242 (83.5%)**	**157 / 157**	**42 / 44**	2.67 s*
v9, all 54 dialogs	250 / 309 (80.9%)	206 / 206	51 / 54	2.75 s*
v10: + persons / actions rules (PROJECTION)	210 / 242 · 266 / 309	157 / 157	—	same

* mean includes the type model's ~1.35 s, paid once per report on the first description; later model turns ~1.7 s; ~60% of turns are answered by the fast path in ~0 ms. Incident type 33 → 37/43 with the router; severity 42/42; location 41/44; time 39/42. On the 10 later dialogs: 9/10 conversations ended right, 49/49 tool calls, 48/67 fields before the v10 rules.

8. Models compared (same code, same day, BOARD)

	Llama 3.2 3B	Gemma 3 4B	Llama + Gemma (type)
fields right / 242	190	192	193 (202 with v9 fixes)
incident type / 43 · persons / 18	33 · 9	**39 · 13**	37 · 8
location / 44 · tool / 157	**41 · 153**	34 · 149	41 · 153
conversations ended right / 44	41	37	**42**
model time per call (mean) · per token	**1.73 s · ~47 ms**	2.60 s · ~59 ms	1.8 s + 1.35 s once

Qwen3 1.7B, Gemma 3 1B and Qwen2.5 0.5B are built and allowed on the board but were not run on the eval.

9. What did not work (dropped by its pre-written rule)

experiment	result (BOARD)
Longer prompt with field definitions (v5)	180/242 vs 190: the model copied a place from an example and mixed lift with equipment failure
Compact answers, fewer output tokens (v6)	faster, but tool accuracy dropped (0.962 vs 0.975)
15-field form	the 10 required fields fell 202 → 189 on the same turns; new fields only 28/83
Answer pre-fill, first version	every answer empty: the pre-filled text split a token (":{"); fixed by ending on a token boundary, re-run pending

10. Limitations and next steps

- **Speech timing on the board** (speech-to-text, text-to-speech, whole spoken turn) is not measured yet; the tools are built (eval/wer.py, eval/turn_time.py). The first turn is the slow one (~3.3 s of model time with the router); the speed switches above target ~2 s.
- **Equipment** (8/20) and **persons** remain the weakest fields; Gemma's better persons score suggests giving it that field too.
- Hindi / Hinglish and the wake word are built and tested offline; their board accuracy is measured against pre-written rules before the demo uses them by default. The locked test split is run once, at the end.